

Informe Trabajo Práctico Final - ALP

Matías Nores

2026

1 Introducción

Este documento servirá para presentar el Trabajo Práctico final que desarrollé para la materia Análisis de Lenguajes de Programación. En el mismo se explica un poco la idea general del TP, su objetivo, implementación, alcances y decisiones de diseño.

2 Idea general y Objetivos

El Álgebra Relacional es un lenguaje para realizar consultas sobre bases de datos relacionales de forma procedimental, es decir, que se debe explicitar tanto los datos deseados como la forma de obtener los mismos (por decir de alguna forma, el "orden de las operaciones"). El objetivo de mi TP final fue el de desarrollar un intérprete interactivo en Haskell para dicho lenguaje de consulta, ofreciendo al usuario la posibilidad de realizar las mismas por medio de las operaciones fundamentales del AR por medio de una CLI tipo MySQL. Estas vendrían a ser:

- **Selección (σ):** retorna las tuplas de una relación que cumplen con un cierto predicado asociado a los atributos
- **Proyección (π):** retorna los atributos deseados de una relación
- **Renombramiento (ρ):** asigna temporalmente un alias a una relación dada, renombrando también sus atributos
- **Unión ($\cup$):** une las tuplas de dos relaciones en una sola, sin duplicados (esta operación está restringida a relaciones que tienen la misma cantidad de atributos, así como igualdad en sus dominios)
- **Diferencia ($-$):** $A - B$ retorna las tuplas de la relación A que no están en B (requiere las mismas restricciones que la unión)
- **Producto cartesiano ($\times$):** retorna todas las tuplas posibles que se pueden formar uniendo las tuplas de ambas relaciones de a pares

Las primeras 3 operaciones son unarias, las otras 3 son binarias. Para ayudar a que sean más legibles y más fáciles de escribir nuestras consultas, el AR proporciona operaciones adicionales que pueden ser escritas en términos de las fundamentales (ie, no agregan poder de cómputo, solo facilitan la escritura) y sirven para expresar consultas específicas. Estas son:

- **Intersección:** retorna una relación con las tuplas que están en ambas relaciones (también exige igualdad de esquemas)
- **Producto natural:** equivale a realizar un producto cartesiano entre dos relaciones, seguido de hacer selección forzando igualdad de atributos en común y luego sacar columnas repetidas
- **División:** $A \div B$ retorna todas las tuplas x tales que (x,y) es una tupla de A para toda tupla y de B

- **Asignación:** no tanto una operación de AR en sí, es más una funcionalidad para asociar resultados de consultas a un nombre. Aunque se la suele considerar parte del AR
- **Agrupamiento:** permite el uso de funciones especiales (COUNT, SUM, MAX, MIN y AVG) para resumir información de muchas filas en un solo valor y agruparlo en uno o más atributos. Esto provee una especie de extensión al Álgebra Relacional para hacer al intérprete menos "teórico"

3 Librerías

A continuación se listarán varias de las librerías de Haskell utilizadas para la realización del TP:

- **Data.List:** usada principalmente para acceder a ciertas funciones de listas que no están en Prelude, como intercalate, transpose, nub, intersect, etc. Útil para muchas operaciones en tablas
- **Data.Map:** bastante central para la definición de tablas y el catálogo
- **Data.Set:** usado para optimizar la evaluación de Proyección
- **Control.Monad:** necesaria para definir las instancias de mónadas de los tipos monádicos nuevos definidos, como ReaderM y Result
- **Control.Applicative:** para definir la instancia de Applicative de dichas mónadas sin que se queje GHC o GHCi
- **System.IO:** para manejar las funcionalidades de entrada y salida del intérprete
- **Text.Parsec:** librería principal para construir el AST del lenguaje
- **Text.ParserCombinators.Parsec:** usado para definir ciertas reglas léxicas básicas del lenguaje y para acceder a algunos combinadores de parsing primitivos
- **Text.Parsec.Language:** fue crucial para definir ciertos tokens y declarar palabras reservadas
- **Text.Parsec.Expr:** fundamental para la construcción automática del parser, en especial en cuanto a la asociatividad y precedencia de las operaciones de AR
- **System.Console.Haskeline:** usado para proveer una interfaz de E/S más cómoda para el usuario, permitiendo backspaces visibles, mover el cursor en la CLI y acceder a comandos previos
- **Control.Monad.IO.Class:** para acceder a funciones que permitan elevar operaciones de E/S (ej: liftIO)
- **Data.Function:** usado para importar algunos combinadores de funciones como "on" ((op 'on' f) x y = f x 'op' f y)

4 Estructura del programa

El código del intérprete está escrito en Haskell, y tiene la siguiente estructura:

- **src:** esta carpeta contiene los archivos principales para la implementación del intérprete. Los mismos son:
 - **Common.hs:** en el mismo se definen tipos de datos de Haskell para representar filas, tablas, atributos, predicados, valores, términos y funciones de agrupamiento. Pero por sobre todo, contiene la definición del AST, tanto para expresiones del Álgebra Relacional como para sentencias más generales (consultas, asignaciones, secuencias).
 - **Parser.hs:** se implementa el parser del lenguaje usando principalmente Text.Parsec y otras funcionalidades para generar automáticamente el mismo (palabras reservadas, asociatividad, precedencia, etc).

- **Eval.hs**: en este archivo se encuentran definidas los tipos de datos monádicos a utilizar para la ejecución del programa capturando ciertos efectos de estado y error. Se implementa en el mismo la evaluación de términos, predicados, expresiones de AR y sentencias.
- **PPAR.hs**: implementa un pretty-printer para mostrar de forma prolija las tablas que resultan de las consultas.
- **app**: esta carpeta contiene un único archivo (**main.hs**) en el cual se implementa una CLI para usar el intérprete de AR y probarlo con una "base de datos" hardcodeada (o sea, un catálogo), la misma BD que se usó en Teoría de Bases de Datos para la práctica (la de proveedores, partes y proyectos). La CLI cuenta también con la posibilidad de consultar el catálogo, posibles operaciones, salir del programa, etc.
- **ejemplos**: en esta carpeta se proveen 2 archivos:
 - **consultas.txt**: contiene consultas y comandos válidos que se pueden usar de modelo para testear el intérprete
 - **error.txt**: las consultas y comandos de este archivo tiran algún tipo de error
- **package.yaml**: usado para resolver dependencias de librerías de Haskell, archivos extra y organización del código fuente a usar, así como también el ejecutable.
- **stack.yaml**: generado automáticamente por el comando de terminal stack init. Define el entorno del programa.
- **stack.yaml.lock**: también generado por stack init, usado para seguridad y consistencia.
- **README.md**: acá encontrarán ciertas cuestiones de cómo compilar, ejecutar y usar el intérprete, así como también algunas consideraciones.

También encontrarán este mismo informe y el enunciado en el cual se basó el desarrollo del Trabajo Práctico.

5 Algunas cuestiones de diseño

A continuación algunas decisiones clave para el desarrollo:

- **Stack**: fue de gran utilidad para construir y gestionar dependencias y versiones de GHC adecuadas para el proyecto, asegurando que pueda ser compilado y ejecutado en diferentes entornos con las mismas versiones de librería con las que fue desarrollado. También provee facilidad al usuario a la hora de compilar, usarlo con GHCI y ejecutar el intérprete.
- **Representando tablas**: el tipo usado para tablas fue [Map Atributo Valor], donde Atributo es un alias de String y Valor puede ser entero, booleano o string (nada de atributos multivaluados). Se optó por lista de mapas para facilitar el acceso a los valores mediante los nombres de atributos, lo cual resultó esencial para evaluar la Selección y Proyección. También flexibilizó los esquemas de tablas si era necesario agregar, quitar o modificar nombre de columnas, así como para simular ciertos comportamientos como eliminar filas duplicadas.
- **Catálogo**: se definió como estado global del programa un mapa de nombres de tabla a tablas en sí (Map NombreRel Tabla). Esto sirvió mucho a la hora de consultar si una tabla existía o si su nombre ya estaba en uso (Map tiene búsqueda de orden logarítmico), lo cual aporta a la comodidad de escribir consultas con nombres de tabla y de modificar el mismo siempre que se haga una operación de asignación.
- **Mónadas**: el programa utiliza 3 tipos de datos monádicos para capturar ciertos efectos computacionales:

- **Result:** esta mónada captura tanto resultados correctos de evaluación como algunos errores típicos de referencia (ej: tabla no existente) y de operación (ej: esquemas no iguales para unión o diferencia), siendo así una forma más descriptiva de capturar y mostrar errores que usando Maybe (donde Nothing no provee información adicional). Al ser instanciada como mónada, Result se asegura de propagar de forma correcta y automática los errores en cualquier nivel de ejecución.
 - **ReaderM:** la usamos para encapsular la evaluación de expresiones en lugar de pasar el catálogo como argumento explícito. Dicha mónada modela un entorno de "solo lectura", lo cual impide modificaciones accidentales del catálogo al evaluar una consulta.
 - **InputT:** usé este transformador de mónada de System.Console.Haskeline para proveer un entorno amigable de I/O, funcionando así como un "wrapper" de terminal. Las operaciones de la mónada IO deben ser elevadas, y se permite entonces editar en la CLI la línea de entrada (por ejemplo para borrar input, mover el cursor, buscar comandos anteriores, etc), siendo así más parecido a un programa de terminal.
- **Parsing:** se optó por usar la librería Parsec ya que la notación permite que el código sirva como su propia documentación. Además, las extensiones como Text.Parsec.Expr permiten facilitar mucho la resolución de ambigüedades al indicar la asociatividad y precedencia de forma declarativa, así como también la definición de palabras y símbolos reservados.
 - **Chequeo de tipos:** se cuenta con un sistema de tipado dinámico fuerte. En las primeras versiones del proyecto los errores de tipo eran silenciosos, ya sea devolviendo una tabla vacía o devolviendo columnas con valores nulos (ej: SUM NOMBRE AS Total, la columna Total resultante quedaba llena de ceros). La verificación de los tipos se llevó a cabo en varias partes del programa, por ejemplo en la comparación de esquemas antes de hacer unión, diferencia o intersección de tablas. El otro nivel de validación se da en el tipo de dato Valor, con lo cual se dan dos posibles constructores (N para enteros, Str para cadenas) y se mejoró la evaluación de predicados y funciones de agregado para reportar errores de tipo. Por ejemplo si se trataba de hacer SELECT (CANT = "Juan") SPJ, o usar funciones de agregado como SUM y AVG sobre otra cosa que no sea N Int (como mencionamos antes). Todo esto aportó mucho a darle seriedad al trabajo sin pasar por alto estos errores.
 - **Pretty-Printer:** el mismo fue pensado para darle a las tablas un estilo similar al usado por programas como MySQL. Cuenta con algoritmos internos para calcular el ancho de cada columna (comparando el nombre de cada atributo con el valor "más largo" que asume en dicha columna) y así luego calcular el espaciado y poner separadores de forma correcta, para que quede prolija y alineada la visualización de las mismas. El pretty-printer en sí convierte las tablas a String para después ser mostradas por pantalla de forma legible. Se optó String por encima de Doc (de Text.PrettyPrint) por simplicidad de código y por no aportar realmente una ventaja significativa, ya que es más importante el problema de alinear de forma estricta.
 - **Operaciones adicionales:** estas son las definidas a partir de las fundamentales. Como dijimos, no agregan potencia al AR pero sí facilidad para escribir consultas. Implementé las mismas de forma directa en el AST por comodidad, ya que algunos casos eran fáciles de pensar como patrones (como la intersección entre A y B, la cual es lo mismo que hacer $A - (A - B)$), pero en otros como división y producto natural dejaba de ser tan trivial. Se optó por implementar también agrupamiento, el cual utiliza ciertas funciones de agregado que permiten obtener información más concisa y resumida que la guardada en las filas (suma, promedio, máximo, mínimo y cantidad) como lo es GROUP BY en SQL.

6 Fuentes y referencias

Las bases teóricas del TP fueron tomadas de apuntes de cátedra y todo lo visto respecto al Álgebra Relacional en la materia Teoría de Bases de Datos, así como la BD que se usa para probar el intérprete. La operación de agrupamiento en AR no fue dada en la materia más que para SQL (GROUP BY), y esto fue de lo más difícil de implementar en su momento.

Fue de gran ayuda para realizar el TP el uso de IA, la cual fue de gran utilidad a la hora de resolver problemas de compilación, debugging, configurar el entorno y formar esqueletos básicos de algunas partes del código que pueden llegar a ser repetitivas o tediosas de escribir a mano. Cabe aclarar que hubo mucha inspiración de mi parte a la hora de definir los tipos de datos a utilizar, en especial los monádicos y el AST del lenguaje.

Se usó en gran medida la documentación de la página oficial de Haskell (haskell.org) para conocer las funciones provistas por cada librería, así como también se recurrió a los otros TPs desarrollados en la materia para más inspiración y cuestiones de diseño del lenguaje.